

Handout: Aesop Tactic

Graduate Seminar on Applied Logic

Yannik Spitzley June 6, 2026

1. What is Aesop?

Aesop stands for **A**utomated **E**xtensible **S**earch for **O**bvious **P**roofs. It is a tactic in Lean4 designed to automatically find proofs for seemingly obvious statements, such as $2 \cdot n$ is even for $n \in \mathbb{N}$ or $A \cap B \subseteq A$ for sets A, B .

- **White-box approach:** The rule sets and parameters are user-specified, which makes the tool transparent, easy to debug and interesting for domain-specific applications.
- **Best-First proof search:** Aesop utilizes a tree-based search procedure, where nodes are expanded according to user-assigned priorities and a best-first strategy.

2. The Search Space & Tree Structure

Aesop searches for proofs by constructing a tree with alternating nodes: **goal nodes** and **rule application (rapp) nodes**.

- **Goals (G):** Each goal has the form $\vec{h} : \vec{T} \vdash U$ for hypotheses $\vec{h}$ and a target U .
- **Rules (R):** A rule is a partial function mapping a goal to a set of generated subgoals. Rules must be provability-reflecting, i.e. if its subgoals are provable, the initial goal is also provable.
- **Node states:** A node in the search tree can have one of three states:

State	Goal node	Rapp node
proved	Has at least one <i>proved</i> child.	All children are <i>proved</i> .
stuck	Can not be <i>proved</i> with the given rules.	
unknown	Neither <i>proved</i> nor <i>stuck</i> .	

3. Rule Types & Search Algorithm

Aesop distinguishes between three specific types of rules:

1. **Normalisation Rules:** Used for simplification (e.g., introducing assumptions, unfolding definitions). They generate ≤ 1 subgoals (no branching) and preserve provability. Their execution order is determined by a user-specified integer priority.
2. **Safe Rules:** Used for safe branching. They can generate arbitrary numbers of subgoals and preserve provability. The priority assigned to a safe rule impacts the search performance.
3. **Unsafe Rules:** They do not necessarily preserve provability, can generate arbitrary subgoals and might lead to stuck branches requiring backtracking. They are assigned a success probability between 0% and 100% to guide the search.

The Main Loop (Best-First Strategy):

The search begins with an initial goal with priority 100%. When a rule is applied, the priority of the subgoals is calculated as $\text{Prio}(G_{\text{sub}}) = \text{Prio}(G) \cdot P_{\text{Success}}(R)$. The algorithm iteratively chooses an unknown goal with the highest priority and expands it in three distinct phases:

- **Phase 1:** Normalisation (Simplifying the goal).
- **Phase 2:** Safe Rules (Safe branching).
- **Phase 3:** Unsafe Rules (Best-first search based on success probabilities).
- If Aesop fails to prove the initial goal, it reports the **safe goals**, i.e. the goals remaining after only applying phase 1 and phase 2.

4. Usage in Lean

Aesop rules are arbitrary tactics. Writing a tactic for every definition/lemma is inconvenient, so Aesop provides **rule builders** to automatically register Lean expressions as rules.

- **apply:** Enables backward reasoning by applying a theorem to reduce the target.
- **forward:** Enables forward reasoning by taking existing hypotheses and adding new logical consequences to the context.
- **simp:** Adds lemmas to a rule set used by Lean’s simplifier during the normalisation phase.
- **constructors:** Creates a multi-rule that generates subgoals for all constructors of a given inductive type.

Implementation:

- Rules can be registered locally for a single tactic call or globally into specific rule sets.
- Call `aesop` to use the tactic or `aesop?` to have Aesop print the proof it found.
- Configuration options allow adding/erasing rules or tweaking the search parameters.

5. Handling Metavariables

A significant challenge in best-first search is the **coupling problem** caused by metavariables.

- **The Problem:** If a rule generates two subgoals sharing a metavariable (e.g. $n \leq ?m$ and $?m \leq k$), the subgoals are not independent. The proof for the first goal determines the instantiation of $?m$, which impacts the provability of the second goal.
- **Aesop’s Solution:** Aesop treats goals with instantiated metavariables as additional subgoals. If an assignment is made (e.g. $?m := a$), the dependent goal is copied along with this assignment into the newly created proof branch.
- **Consequences & Best Practices:** Because this copying mechanism creates a large search space, rules generating metavariables should only be registered locally. Furthermore, if a *safe* rule assigns a metavariable, Aesop treats it as an *unsafe* rule with a 90% success probability as fixing a metavariable restricts the search is almost never safe.

References

- [LF23] Jannis Limperg and Asta Halkjær From. “Aesop: White-Box Best-First Proof Search for Lean”. In: *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*. 2023, pp. 253–266.
- [Ll26] Jannis Limperg and leanprover-community. *Aesop: Automated Extensible Search for Obvious Proofs*. <https://github.com/leanprover-community/aesop>. Accessed on: June 6, 2026. 2026.